



lab_FSM

CT Praktikum: Finite State Machine (Liftsteuerung)

1 Vorbereitung

Studieren Sie die Vorlesungsfolien zu «Software State Machines» und das abgegebene Beispiel (stateEventImpExample.pdf). Orientieren Sie sich bei der Lösung der kommenden Aufgaben an der Struktur mit **Event_handler** <get_event()>, **Action_handler** <led_turn_off()>, **led_turn_on()** und **State Machine** <fsm_handle_event()>.

2 Funktion

In diesem Praktikum implementieren Sie eine Finite State Machine (FSM) als Steuerung für eine einfache Liftanlage.

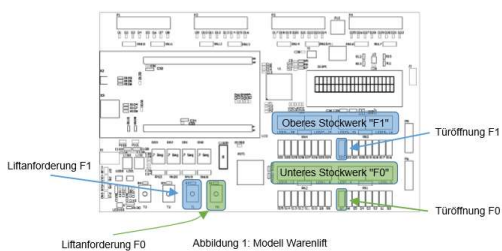
Das Modell in **Abbildung 1** zeigt den CT-Board-Warenlift, der den Keller eines Restaurants (untere LED-Reihe) mit der Küche (obere LED-Reihe) verbindet. Die **Taster T0** und **T1** dienen als **Lift-Anforderungstaster**, die **Schalter S7** und **S23** als **Türöffnungsrichtungen** auf den beiden Stockwerken.

Als erstes kontrollieren Sie das Öffnen und Schliessen der Lifttüren und implementieren die Fahrt zwischen den beiden Stockwerken.

Erweitern Sie dann das Basismodell um Sicherheitspausen vor der Fahrt, eine Signalsteuerung im oberen Stock sowie eine Gewichtskontrolle für die Beladung im Keller.

3 Lernziele

- Sie können eine Funktionsbeschreibung in ein einfaches UML-State-Diagramm umsetzen.
- Sie können ein UML-State-Diagramm in C implementieren.



30.01.2026

lab_FSM.docx

1

4 Aufgaben

Die Funktionen der Liftsteuerung werden schrittweise umgesetzt.

4.1 Türkontrolle

In einem ersten Schritt soll eine einfache FSM gemäss **Abbildung 2** für den Türöffnungsmechanismus im Keller (F0) implementiert werden:

- Studieren Sie die Funktion **ah_get_event()** im Modul **event_handler**. Wie werden die beiden Events **EV_DOOR0_CLOSE_REQ** und **EV_DOOR0_OPEN_REQ** generiert?
- Implementieren Sie die Funktionen **fsm_init()** und **fsm_handle_event()** im Modul **state_machine** gemäss dem Diagramm in **Abbildung 2**.

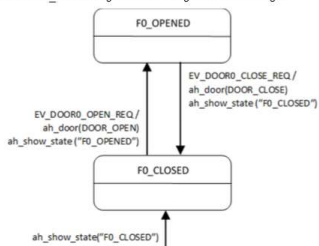


Abbildung 2: UML-State-Diagramm 'Türsteuerung'

Hinweise: Das Modul **action_handler** enthält eine Funktion **ah_show_state(char text[])**. Mit dieser Funktion können Sie Strings für das Debugging auf den LCD ausgeben. Zeigen Sie damit bei einer Transition den Folgezustand an, zum Beispiel: **ah_show_state("F0_CLOSED");**
Die Aufrufe **ah_door(DOOR_OPEN)** und **ah_door(DOOR_CLOSE)** dienen der Animation des Warenlifts.

30.01.2026

lab_FSM.docx

2

S7 wird eingeschalten -> Positive Flanke -> EV_DOOR0_OPEN_REQ
S7 wird ausgeschalten -> Negative Flanke -> EV_DOOR0_CLOSE_REQ
Tür F0 öffnen = S7 einschalten, Tür F0 schliessen = S7 ausschalten.

```
1 /* Detect dip switch events */
2 uint16_t dip_value = CT_DIPSW_WORD0;
3 uint16_t dip_value_pos = ~dip_value & dip_value;
4 uint16_t dip_value_neg = dip_value & ~dip_value;
5 dip_value_pos = dip_value;
6
7 if (dip_value_pos & DOOR0_MASK) {
8     return EV_DOOR0_OPEN_REQ;
9 }
10 else if (dip_value_neg & DOOR0_MASK) {
11     return EV_DOOR0_CLOSE_REQ;
12 }
13 else if (dip_value_pos & DOOR1_MASK) {
14     return EV_DOOR1_OPEN_REQ;
15 }
16 else if (dip_value_neg & DOOR1_MASK) {
17     return EV_DOOR1_CLOSE_REQ;
18 }
```

```
1 // STUDENTS: To be programmed
2 #define TEXT_F0_SAFETY_PAUSE "F0_SAFETY_PAUSE"
3 #define TEXT_F1_SAFETY_PAUSE "F1_SAFETY_PAUSE"
4 // END: To be programmed
5
6 #include <stdio.h>
7
8 // STUDENTS: To be programmed
9 #define TEXT_F0_SAFETY_PAUSE "F0_SAFETY_PAUSE"
10 #define TEXT_F1_SAFETY_PAUSE "F1_SAFETY_PAUSE"
11 // END: To be programmed
12
13 void fsm_init(void)
14 {
15     action_handler_init();
16     ah_show_exception(NORMAL, "");
17     /* go to initial state & do initial actions */
18     // STUDENTS: To be programmed
19     ah_door(DOOR_CLOSE);
20     ah_show_state(TEXT_F0_CLOSED);
21     // END: To be programmed
22 }
```

- Initialisiert die FSM
- Türe zu Beginn geschlossen und F0_Closed wird angezeigt

```
1 void fsm_handle_event(event_t event)
2 {
3     // STUDENTS: To be programmed
4     switch (event)
5     {
6         case EV_DOOR0_OPEN_REQ:
7             state = F0_OPENED;
8             ah_door(DOOR_OPEN);
9             ah_show_state(TEXT_F0_OPENED);
10            break;
11
12            case EV_DOOR0_CLOSE_REQ:
13                state = F0_CLOSED;
14                ah_door(DOOR_CLOSE);
15                ah_show_state(TEXT_F0_CLOSED);
16                break;
17
18            default:
19                break;
20
21            case EV_BUTTON_F1:
22                state = F1_SAFETY_PAUSE;
23                ah_door(DOOR_LOCK);
24                ah_show_state(TEXT_F1_SAFETY_PAUSE);
25                timer_start(SAFETY_DURATION);
26                break;
27
28            default:
29                break;
30
31            break;
32    }
```

- F0_OPENED**
 - Event: EV_DOOR0_OPEN_REQ
 - Aktion: Türschliessen
 - Neuer State: F0_CLOSED
- F0_CLOSED**
 - Event: EV_DOOR0_OPEN_REQ
 - Aktion: Türöffnen
 - Neuer State: F0_OPENED
- F0_CLOSED**
 - Event: EV_BUTTON_F1
 - Aktion: Türverriegeln, Timer starten
 - Neuer State: F1_SAFETY_PAUSE

4.2 Liftfahrt

Im zweiten Schritt wird die Liftfahrt zwischen Keller und Küche ("F0") gemäss **Abbildung 3** implementiert, ebenso der Türöffnungsmechanismus im oberen Stock.

- Sehen Sie in der Funktion `eh_get_event()` nach, wie die Events `EV_BUTTON_F0` und `EV_BUTTON_F1` generiert werden. Sie dienen dazu, die Fahrt auszulösen, **falls die Türen geschlossen sind**.
- Die Events `EV_F0_REACHED` und `EV_F1_REACHED` signalisieren, wann der untere resp. der obere Stock erreicht sind und der **Liftmotor abgeschaltet werden muss**. Diese Events werden **automatisch** von der Lift-Animation im Rahmenprojekt erzeugt.
- Erweitern Sie die State Machine so, dass die zusätzlichen States und Transitionen unterstützt werden.
- Funktioniert Ihr Lift ohne Crash?

Ja, Lift startet bei geschlossener Türe, Beim Zielstock -> Motor ausgeschaltet, Danach Door entriegelt default für ungültige Events

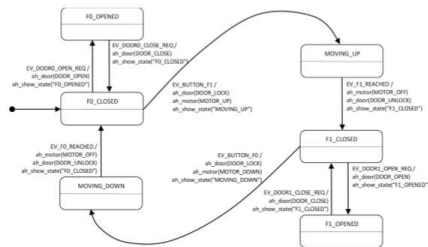


Abbildung 3: UML-State-Diagramm 'Liftfahrt'

30.01.2026

lab_FSM.docx

3

```
case MOVING_UP:
    switch (event)
    {
        case EV_F1_REACHED:
            state = F1_CLOSED;
            ah_motor(MOTOR_OFF);
            ah_door(DOOR_UNLOCK);
            ah_show_state(TEXT_F1_CLOSED);
            break;

        default:
            break;
    }
    break;
case MOVING_DOWN:
    switch (event)
    {
        case EV_F0_REACHED:
            state = F0_CLOSED;
            ah_motor(MOTOR_OFF);
            ah_door(DOOR_UNLOCK);
            ah_show_state(TEXT_F0_CLOSED);
            break;

        default:
            break;
    }
    break;
default:
    break;
}
// END: To be programmed
```

MOVING_UP

- Await auf `EV_F1_REACHED`
- Motor aus: `ah_motor(MOTOR_OFF)`
- Tür entriegeln: `ah_door(DOOR_UNLOCK)`
- neuer State: `F1_CLOSED`

MOVING_DOWN

- wartet auf `EV_F0_REACHED`
- Motor aus: `ah_motor(MOTOR_OFF)`
- Tür entriegeln: `ah_door(DOOR_UNLOCK)`
- neuer State: `F0_CLOSED`

4.3 Komfortfunktionen

Erweitern Sie nun das Liftmodell um **mindestens eine** der folgenden 3 Funktionen:

a) Kurze Sicherheitspause vor der Fahrt

Bevor der Lift nach oben oder nach unten losfährt, soll eine Sicherheitspause von 1.5s eingelegt werden, während der die Türen zwar verriegelt sind (`DOOR_LOCK`), aber der Lift noch stillsteht. Benutzen Sie das vorgegebene Modul `timer`, um die notwendigen Verzögerungen zu implementieren, konsultieren Sie wiederum `eh_get_event()`, um herauszufinden, wie das zugehörige Event heisst, das Sie empfangen, wenn die Zeit abgelaufen ist.

Erweitern Sie Ihr Programm um die fehlenden Funktionen:

- Erweitern Sie das UML-State-Diagramm und tragen Sie die neuen Übergänge ein.
- Rufen Sie in `main.c` einmalig die Funktion `timer_init()` auf, um den verwendeten Timer zu initialisieren.
- Den Timer können Sie mittels der Funktion `timer_start(duration)` starten, und mittels `timer_stop()` stoppen, falls der Timer abgebrochen werden soll. Der Funktionsparameter `duration` bezieht sich auf die Anzahl Perioden eines 100 Hz Signals (10ms-Schritte).

```
#define SAFETY_DURATION 150 // 150 * 10ms = 1.5s
#define SIGNAL_DURATION 100 // 100 * 10ms = 1s
```

b) Signalisation, wenn der Lift im oberen Stock ankommt

Um das Küchenpersonal darauf aufmerksam zu machen, dass eine Lieferung aus dem Keller bereit steht, soll ein Blinklicht implementiert werden, das losgeht, wenn der Lift den oberen Stock erreicht, und stoppt, wenn die Lifttüre geöffnet wird.

Erweitern Sie Ihr Programm folgendermassen:

- Erweitern Sie das UML-State-Diagramm und tragen Sie die neuen Übergänge ein.
- Auch hier wird ein Timer benötigt, beachten Sie dazu die Angaben oben unter a).
- Um das Blinklicht an- und abzuschalten, verwenden Sie die Aufrufe `ah_signal(SIGNAL_ON)` und `ah_signal(SIGNAL_OFF)`. Diese Funktionen steuern lediglich die Signallampe; das Blinken müssen Sie selber mit entsprechenden States implementieren.

Timer init in Main:

```
22 #include "timer.h"
23
24
25 // ...
26
27 // ...
28
29 // ...
30
31 // ...
32
33 // ...
34
35 // ...
36
37 // ...
38
39 // ...
40
41 // ...
42
```

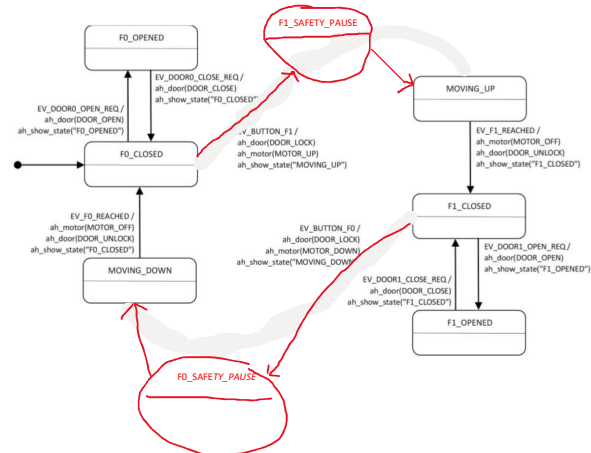
```
break;
case F0_SAFETY_PAUSE:
    switch (event)
    {
        case EV_TIMEOUT:
            state = MOVING_DOWN;
            ah_motor(MOTOR_DOWN);
            ah_show_state(TEXT_MOVING_DOWN);
            break;

        default:
            break;
    }
    break;
case F1_SAFETY_PAUSE:
    switch (event)
    {
        case EV_TIMEOUT:
            state = MOVING_UP;
            ah_motor(MOTOR_UP);
            ah_show_state(TEXT_MOVING_UP);
            break;

        default:
            break;
    }
    break;
```

- F0_SAFETY_PAUSE**
- Await `EV_TIMEOUT`
 - Motor nach unten: `ah_motor(MOTOR_DOWN)`
 - neuer State: `MOVING_DOWN`

- F1_SAFETY_PAUSE**
- Await `EV_TIMEOUT`
 - Motor nach oben: `ah_motor(MOTOR_UP)`
 - neuer State: `MOVING_UP`



30.01.2026

lab_FSM.docx

4

c) Gewichtskontrolle beim Beladen im unteren Stock

Es ist nun so, dass der Warenlift aus Sicherheitsgründen nicht mehr als 50 Kg transportieren darf. Wenn also der Lift im Keller ist und die Lifttüre offensteht, soll das Gewicht der Ladung angezeigt und überprüft werden, bei zu hohem Gewicht darf nicht losgefahren werden. Als Gewichtseingabe wird das CT-Board Potentiometer verwendet, das so konfiguriert ist, dass Werte zwischen 0 und 63 erzeugt werden. Das aktuell eingestellte Gewicht sehen Sie auf der 7-Segment-Anzeige.

Erweitern Sie Ihr Programm wie folgt:

- Erweitern Sie das UML-State-Diagramm und tragen Sie die neuen Übergänge ein.
- Rufen Sie in `main.c` einmalig die Funktion `eh_init()` auf, um den verwendeten ADC-Wandler zu initialisieren.
- Verwenden Sie `eh_weight_control(WC_ENABLE, 50)`, um die Waage einzuschalten und die zugehörigen Events `EV_WEIGHT_TOO_HIGH` sowie `EV_WEIGHT_OK` zu empfangen. `eh_weight_control(WC_DISABLE, 0)`, schaltet die Waage wieder aus.
- Um eine Warnung anzuzeigen, verwenden Sie `ah_show_exception(WARNING, "Gewicht zu hoch!")`, und um die Warnung zu löschen `ah_show_exception(NORMAL, "")`.

5 Bewertung

Die lauffähigen Programme müssen präsentiert werden. Die einzelnen Studierenden müssen die Lösungen und den Quellcode verstanden haben und erklären können.

Bewertungskriterien	Gewichtung
Die Türkontrolle ist funktionsfähig.	1/4
Der Lift fährt fehlerfrei	1/4

c) Gewichtskontrolle beim Beladen im unteren Stock

Es ist nun so, dass der Warenlift aus Sicherheitsgründen nicht mehr als 50 Kg transportieren darf. Wenn also der Lift im Keller ist und die Lifttüre offensteht, soll das Gewicht der Ladung angezeigt und überprüft werden, bei zu hohem Gewicht darf nicht losgefahren werden.

Als Gewichtseingabe wird das CT-Board Potentiometer verwendet, das so konfiguriert ist, dass Werte zwischen 0 und 63 erzeugt werden. Das aktuell eingestellte Gewicht sehen Sie auf der 7-Segment-Anzeige.

Erweitern Sie ihr Programm wie folgt:

- Erweitern Sie das UML-State-Diagramm und tragen Sie die neuen Übergänge ein.
- Rufen Sie in main.c einmalig die Funktion eh_init() auf, um den verwendeten ADC-Wandler zu initialisieren.
- Verwenden Sie eh_weight_control(WCTL_ENABLE, 50), um die Waage einzuschalten und die zugehörigen Events EV_WEIGHT_TOO_HIGH sowie EV_WEIGHT_OK zu empfangen.
eh_weight_control(WCTL_DISABLE, 0), schaltet die Waage wieder aus.
- Um eine Warnung anzuzeigen, verwenden Sie ah_show_exception(WARNING, "Gewicht zu hoch!"), und um die Warnung zu löschen ah_show_exception(NORMAL, "").

5 Bewertung

Die lauffähigen Programme müssen präsentiert werden. Die einzelnen Studierenden müssen die Lösungen und den Quellcode verstanden haben und erklären können.

Bewertungskriterien	Gewichtung
Die Türkontrolle ist funktionsfähig.	1/4
Der Lift fährt fehlerfrei.	1/4
Sie haben mindestens eine Komfortfunktion implementiert.	2/4